

Mikhail LeTal: a chess agent for AI Chessathon 2026

Design, evidence and rating estimate

Alexander Urvancev

Yan Beletskiy

September 2026 (build v1.0, the compiled engine; updated with each build)

Abstract

Mikhail LeTal is a chess agent written for AI Chessathon 2026 by a two-person team. The shipped build (v1.0, “Stage 1”) carries its own board representation, move generation, evaluation and search, compiled to machine code by numba at import; python-chess remains in the safety wrapper as the legality oracle and the fallback. Every line of it is our own and is described here so that it can be explained. The engine it replaced — the same algorithms in plain Python on a python-chess board — still ships inside the zip, because it is the readable reference and the oracle the compiled engine is tested against, integer for integer.

The compiled build beats that predecessor by **+560 Elo** (95 % interval +495 to +660; 300 games, $+279 \pm 19$, $96.2\% \pm 1.6\%$) (source: RESULTS.md, row **numba-vs-v0.2-10s**), reaching depth 10–14 in three seconds where it reached 6–10, at 691–913 thousand nodes per second against 50–69 thousand (source: RESULTS.md, “Stage 1 phase 2” tables). It has been validated on the competition platform, where it is measured at **depth 10–12** and 290–569 thousand nodes per second on the slower competition core (source: CALIBRATION.md, 2026-09-08, v1.0 validation). Against rating-limited Stockfish at the full time control it scores 84.4 %, 75.0 %, 65.6 % and 56.2 % at UCI_Elo 1800, 2000, 2200 and 2400, 16 games each (source: RESULTS.md, rows **v1.0-vs-sf*-real**); those brackets are wide and no CCRL-scale rating is claimed for this build (Section 11).

No game in any of those matches ended in a crash, an illegal move, a flag fall or a start-up failure. Every number in this document is a measurement quoted from docs/RESULTS.md or docs/CALIBRATION.md, or a constant quoted from the code and its provenance files; where a figure is an estimate rather than a measurement it says so. Two measurements remain open and are marked in red throughout.

1 The competition and what was built

Format. AI Chessathon 2026 is a competition for chess agents written by student teams. A team uploads a zip whose root `agent.py` exposes one function, `get_move(fen, time_left_ms)`, returning a move in UCI notation (for example `e2e4`, or `e7e8q` for a promotion). The platform plays the latest build that passed validation on a rated ladder (hourly rounds, 08:00–22:00 London). The ladder only seeds a 13-round Swiss tournament over locked builds on the afternoon of Friday 11 September; the Swiss decides qualification for the London final on Saturday 12 September, where each finalist team walks a judge through how its agent was built. Uploads close at 11:00 London on 11 September.

Clock and machine. Each side has 120 s plus 0.5 s per move, measured in wall time by the runner from the moment it writes the request to the moment it reads the reply, so process wake-up, FEN parsing and board construction are on our clock. The agent runs as one process per game, suspended while the opponent thinks, on one core of an AMD EPYC 9V74 at 2.60 GHz with 2 GB of RAM, no network and no GPU; it gets 90 s at start-up before the clock runs.

The Python environment is fixed: Python 3.12 with `python-chess 1.11.2`, `numpy 2.5.2`, `numba 0.67.0`, `torch 2.13.0` (CPU) and `onnxruntime 1.29.0`; nothing else can be installed and native binaries are rejected. A game is lost outright on an illegal move, a crash, running out of memory, missing the 90 s start-up budget, or a flag fall (unless the opponent cannot mate, which is a draw). Draws follow FIDE rules and the referee claims threefold repetition and fifty-move draws automatically; a game that reaches 600 plies (half-moves, counting the opening position) is a draw. Every game starts from a curated near-level opening position, not from the initial position, and the set of openings is not published (source: `brief`, §3; `AGENTS.md`).

What was built. The shipped build is v1.0, Stage 1 of a staged plan (Section 3). It ships `agent.py` (the wrapper) and the package `mikhail_letal`: the compiled board, evaluation and search (`fastboard`, `fasteval`, `fastsearch`), the Python reference engine they are tested against (`evaluation`, `search`, `searchboard`), and the timing, game-state, warm-up and fallback modules — plus `weights/pst.json` and `weights/PROVENANCE.json`. Fourteen files, 85 925 bytes zipped and 269 589 unzipped, against a 50 MB cap. On the development machine it reaches depth 10–14 with selective depth 26–30 in three seconds, at 691–913 thousand nodes per second (source: `RESULTS.md`, “Stage 1 phase 2” tables); on the competition core, depth 10–12 (source: `CALIBRATION.md`, 2026-09-08, v1.0 validation).

Our result. Local: 200/200 against `baselines/random` by checkmate with zero failure terminations, and +560 Elo over 300 games against the previous shipped build (Section 10 and the v1.0 addendum). Against rating-limited Stockfish 19 at the real clock, 16 games per level: 84.4 % at UCI_Elo 1800, 75.0 % at 2000, 65.6 % at 2200 and 56.2 % at 2400, no flag at any level (source: `RESULTS.md`, rows `v1.0-vs-sf*-real`). The predecessor v0.1 cleared the brief’s Stage 0 bar — a score of at least 90 % against `baselines/minimax` with zero flags — at 98.8 % (± 2.4 %) over 40 real-clock games (source: `RESULTS.md`, row `v0.1-vs-minimax-real-clock`), and the solo per-move pass in Section 10 shows the time budget holding, with a worst overshoot of 2 ms.

On the platform: v1.0 passed validation, reaching depth 10–12 in both smoke games with every warm-up phase completed (source: `CALIBRATION.md`, 2026-09-08, v1.0 validation).

The ladder rating, and why little should be read into it. At the snapshot taken at 19:44 on 8 September, v1.0 stood at a provisional ladder rating of **1669** from **five** rated games, won 3, drawn 1, lost 1 (source: `data/cache/leaderboard/qualifier.html`). Of the 401 teams in that snapshot, **156 are rated above it**, which places it 157th — the leaderboard carries no rank field, so that position is derived by sorting on rating and the count above is the figure actually read. The leaders hold 2573–2763 on **48–65** games. Five games is an unconverged seed rather than a rating, and the snapshot was already stale when it was taken: two further rated games (rounds 74 and 75) have since been played and are not in it. The ladder also decides nothing on its own — it seeds the 13-round Swiss on 11 September, and only the Swiss counts for qualification. The number is recorded here because a report that omits its own worst-looking figure is not a record; it is not offered as a measurement of strength, and the Stockfish brackets in Section 11 are the better estimate.

2 Rules-compliance statement

- No third-party engine, wrapper, port or translation is shipped or consulted at runtime. The libraries used at runtime are `python-chess 1.11.2`, `numba 0.67` and `numpy 2.5`, all part of the platform’s fixed stack. `python-chess` supplies board representation and the rules in the safety wrapper; `numba` and `numpy` supply compilation and array storage. None of

them supplies chess knowledge: the board representation, move generation, evaluation and search that play the game are ours.

- **No native binary ships.** numba compiles from the Python source in the zip, at import, on the platform’s own machine. What is uploaded is source a judge can read, which is what the rules require; nothing is precompiled, and `cache=True` is not used — the platform’s filesystem is read-only apart from `/tmp`, which is wiped between games, so a cache would buy nothing.
- No published network is shipped; this build ships no network at all. No parameter that ships was fitted to positions labelled by another engine. Texel tuning was attempted and rejected on measurement, so the evaluation that ships is the untuned parametric prior (Section 6).
- No lookup database of another engine’s moves or evaluations ships in the zip. There is no opening book and no tablebase in this build (Section 8).
- Memorised-code risk. An engine written with a language model can reproduce code or tables it has seen, and Sunfish, whose tables are famous, is a house bot on this ladder. Three measures were taken. (1) Every evaluation table is generated by `tools/gen_pst.py` from a parametric geometric prior (Section 6): each entry is a formula of the square’s geometry times a named parameter, so the numbers are ours by construction and re-running the script reproduces them bit for bit. (2) Search and evaluation were written against a module contract (`docs/DESIGN.md`) from textbook descriptions of the algorithms, and a review pass looked for surface overlaps with known engines: the one found was the class-and-method pair `Searcher.search`, which Sunfish also uses; our class was renamed `Engine` even though everything behind it differs (source: DECISIONS.md, 2026-09-07 item F). (3) The generated tables were compared mechanically against the published tables of **Sunfish**, the Chess Programming Wiki’s “**simplified evaluation function**”, **Rustic Alpha 3**, **TSCP** and **VICE**: whole-table comparisons and affine comparisons (does any table of ours equal a published one after multiplying by a constant and adding a constant?) were all clean, and a row-by-row comparison found exactly **one coincidental row**: the endgame pawn table’s seventh rank is 50 in both ours and the CPW table, because our formula gives $14 \times 5^2 / 7 = 50$ there (source: DECISIONS.md, 2026-09-07 item I).
- Local measuring instruments. A rating-limited Stockfish (path read from the environment variable `YARDSTICK_ENGINE`) and the real Sunfish are used only as local sparring partners for the rating estimate (Section 11); they live outside the repository, never enter the zip, never influence a move, and their source was not read to inform the design.
- Provenance. Every constant that ships has a row in Appendix A, copied from `docs/PROVENANCE.md` and `weights/PROVENANCE.json`, which itself ships in the zip.
- Everything shipped is plain, type-annotated Python that passes `ruff check` and `mypy -strict` (source: INTEGRATION_NOTES.md, “Checks run”). No generated table ships without its generating script; nothing is minified or obfuscated.

3 Architecture and why

The plan has stages, and each stage replaces the previous shipped build only after beating it under the real clock with a 95 % interval above zero. Three architectures were on the table.

1. **python-chess board + our own search and evaluation in pure Python** (Stage 0, builds v0.1 and v0.2). python-chess is a proven implementation of the rules, so correctness risk is confined to our own code: search, evaluation, time management. This is what the first priority of the brief (never lose a game for free) demands from the first session. It is no longer the shipped architecture, but it still ships inside the zip as the reference implementation and as the oracle the compiled engine is tested against (Section 10).

2. **Own board representation, move generation, evaluation and search compiled with numba** (Stage 1, **this build**), with python-chess kept in the wrapper as the legality oracle and fallback. numba compiles at every game start, so the compile time is paid at import, inside the 90 s start-up budget and before the clock starts: measured at 18.5 s from the extracted zip (source: RESULTS.md, row “import + compile, extracted zip”). The root of the iterative-deepening loop is deliberately left in Python; only **negamax** crosses into compiled code, once per root move per iteration. Compiling the root as well was measured and rejected: it raised total compile time to 34.2 s for no gain, because the root executes a handful of times per move rather than millions (source: RESULTS.md, rows “compile time if the root is compiled too” and “Python → jitted call boundary”).
3. **A small self-trained NNUE-style network** evaluated inside the jitted search (Stage 4, expected to be skipped): **onnxruntime** cannot be called once per node from a jitted search, training data would have to be produced first, and the operator would rather explain a tuned hand-written evaluation than a rushed network.

A fourth option, jitting only the evaluation under a python-chess board, was rejected by arithmetic on the measured numbers: at 50–61 thousand nodes per second one node costs roughly 16–20 μ s, of which **evaluate()** is 3.4–3.8 μ s in the middlegame and 1.6 μ s in the endgame (source: RESULTS.md, row “**evaluate()** cost”), so the evaluation is at most a fifth of the node cost and numba cannot read python-chess objects without a conversion that would cost more than it saves. The speed has to come from replacing the board and the move generator, which is Stage 1.

architecture	node rate	depth at 120 s + 0.5 s	import time	peak RSS	status
Stage 0: python-chess + pure Python	50–61k nps (middlegame)	5–6 (seldepth 18–32)	0.033 s	42.4 MB after a 10 s search; 120 MB at the TT cap	superseded (v0.1, v0.2)
Stage 1: numba engine	691–913k nps	10–14 at 3 s (seldepth 26–30)	18.5 s from the zip, against a 90 s budget	349 MB after three moves, against a 2 GB cap	shipped (v1.0)
Stage 4: NNUE-style net	not built	—	—	—	skipped

Table 1: Architectures and the measured numbers that decided the order. Stage 0 rows: RESULTS.md, rows “node rate, middlegame”, “depth at 120 s clock”, “import time”, “peak RSS after a 10 s search”, “RSS at the TT cap”. Stage 1 rows: RESULTS.md, “Stage 1 phase 2” tables, three positions at a three-second budget on the same machine as the Stage 0 rows. “nps” is nodes per second; “seldepth” is the deepest ply reached including quiescence; RSS is resident memory. Both node rates are dev-box figures: the competition core is slower, and Section 7 gives the measured factor.

Under Stage 0, import time was measured three times by timing **import agent** with **time.perf_counter()** in a fresh interpreter and was 0.033 s each time, including a depth-2 warm-up search that runs at import so every code path has executed before the first real move (source: RESULTS.md, row “import time”). After the 2026-09-07 review the node rate on a 3 s search from the initial position rose from 51.7k to 67.2k nps (+30 %, same move **g1f3**, depth 6/17) (source: RESULTS.md, row “review nps”); Section 5 lists what changed.

What import means once the engine is compiled. Under Stage 1 that 0.033 s becomes 18.5 s, and the warm-up stops being a formality. numba compiles a function the first time it is called with a given argument type, and it cannot be interrupted: a function left uncompiled until the game is running would compile *inside* a search, blowing straight through the hard

deadline that protects the clock. So every jitted function is listed in its module and called once at import, on positions chosen to trigger the same type signatures the game will, and `tests/test_fasteval.py` asserts that no new signature appears afterwards under load — a second signature would mean a compilation deferred onto the clock. The warm-up also carries a wall-clock deadline, so that an unexpectedly slow container degrades into a shallower first move rather than losing every game: if the budget runs out, the remaining compilation is finished on the first real move, on the fullest clock of the game. In v1.0 that finish is itself bounded, by `cold_finish_fraction` of the clock.

That bound was later found not to hold. The budget is re-armed for the cold finish, and re-arming resets the slowdown measured during the import — the very number that made the import skip phases in the first place — so the remaining work was predicted at development-machine speed on a machine already known to be slower. The bound has been removed rather than repaired: everything left is compiled unconditionally, and the cost is subtracted from the clock before the move’s budget is computed, so the search that follows shrinks to fit it. Between an accounted cost and an unaccounted one, the accounted one is preferable — a phase deferred instead would compile inside `ENGINE.search`, where numba cannot be interrupted and no deadline applies. **That change is not in v1.0**, the build this report describes; it is recorded here because the bound v1.0 ships with is weaker than it appears.

4 Board representation and move generation

What the shipped build uses. v1.0 has its own board and its own move generator: `mikhail_letal/fastboard.py`, a **0x88 mailbox** representation compiled by numba. A 0x88 board stores the 64 squares in a 128-entry array whose spare half makes the off-board test a single bit operation — `square & 0x88` is non-zero exactly when a square is off the edge — which removes the bounds checking that dominates a naive mailbox generator and suits numba, where an array index is cheap and an exception is not. Moves are packed into integers rather than objects, positions are identified by our own Zobrist keys (`fastboard.position_key`), and `make/unmake` restores the position in place rather than copying it. Correctness is established by `perft` and legal-move-set agreement against `python-chess`, described in Section 10; because the generator is ours, that agreement is a real test rather than a tautology.

`python-chess` remains in `agent.py`, where it converts the incoming FEN, supplies the legality check that every returned move passes, and provides the fallback. So the rules of chess are still enforced by the library at the boundary, and the speed comes from our own representation inside it.

What Stage 0 used, and why the two are described together. The interpreted engine, which still ships as the reference implementation and the oracle, uses `python-chess`’s `Board` throughout: a bitboard representation (one 64-bit integer per piece type and per colour) with fully legal move generation, written and tested by the library’s authors and pinned by the platform at version 1.11.2. Quiescence uses `Board.generate_legal_captures()` so that only captures are generated at the leaves. Squares are indexed as `python-chess` does (`a1 = 0`, `h1 = 7`, `a8 = 56`); evaluation tables are stored from White’s point of view and a Black piece on square s reads the table at $s \oplus 56$ (a vertical mirror). Positions are identified by `python-chess`’s own repetition key, `Board._transposition_key()`, a tuple of the piece bitboards, the side to move, the castling rights and the en passant square; it is exactly what `Board.is_repetition` uses, so what we count as a repetition is what the referee counts (source: `DECISIONS.md`, 2026-09-06, “Position keys”). Two costs are accepted knowingly: the key is a private name (an upgrade of the library could remove it, but the platform pins 1.11.2), and it is a tuple rather than a Zobrist hash. The

compiled board replaces it with its own Zobrist keys, so nothing on the path that plays a rated move depends on a private name.

Perft. A perft test counts the leaf nodes of the full move tree to a fixed depth and compares the count with a reference; it is the standard proof that a move generator is correct. It did not apply to Stage 0, which had no move generator of its own: legality agreement with the referee, which also used python-chess, held by construction. It applies squarely to the shipped build, whose generator is ours, and Section 10 gives the four gates that establish it. The evidence that the surrounding code uses the library correctly is the property-test set (each of the following returns a legal move within budget: all four promotion pieces, capture-promotions, under-promotion to a knight when it is the only mate, en passant for both colours, castling both ways for both colours, a stalemate trap, insufficient material, a single legal move, mate in one, halfmove clock 99, a would-be third repetition) and the 500 fuzz games with zero illegal moves (Section 10) (source: tests/test_properties.py; RESULTS.md fuzz rows). The Stage 1 move generator will be perft-tested against python-chess to depth 4 on the standard public positions and to depth 3 on every collected opening and on at least 2000 random playout positions, plus a legal-move-set equality test on at least 100 000 random positions, before it replaces the library (source: brief, §5 Stage 1 DoD); that table will be added here.

5 Search

The algorithm in one paragraph. The search is a *negamax* tree search: every score is measured from the side to move's point of view, so the value of a position is the maximum over moves of the negated value of the resulting position. *Alpha-beta pruning* carries a window (α, β) of scores that can still matter to the ancestors and abandons a child whose value falls outside it. *Iterative deepening* runs the search to depth 1, then 2, then 3, and so on until the clock says stop; each iteration seeds the next with good first-try moves, and a fully searched answer is always available. At the leaves a *quiescence* search continues with captures (and queen promotions) only, because a static evaluation in the middle of an exchange is meaningless; the side to move may always *stand pat*, that is, accept the static evaluation instead of capturing. A *transposition table* (TT) remembers, per position key, the depth, score, score type (exact, lower bound or upper bound) and best move of an earlier visit, so that positions reached by different move orders are not searched twice. Scores are integers in centipawns (hundredths of a pawn); a mate delivered p plies from the root scores $100\,000 - p$, so shorter mates score higher. All of this is in `mikhail_letal/search.py`.

Why these techniques and not others here. The platform gives one slow core and pure Python gives tens of thousands of nodes per second, so the tree is small (about 10^5 – 10^6 nodes per move at the real clock) and every technique that makes a small tree go deeper, or that keeps the clock check honest, matters more than it would at millions of nodes per second. Null-move pruning, late-move reductions, futility pruning, aspiration windows and quiescence delta pruning were deliberately absent from Stage 0, so that each could be added as a separate experiment rather than as an undifferentiated block (source: DECISIONS.md, 2026-09-06, “No null-move pruning”). All of them were added in v0.2 and all of them ship in v1.0; the v0.2 addendum lists the constants each was given, and the combined effect was to cut the nodes needed to reach depth 6 from the standard start from 238 000 to 17 000.

Table 2: Search techniques as introduced in build v0.1.0, and the local evidence for each; the algorithms are unchanged in the compiled engine, which is a port rather than a redesign. The effects listed are node-rate, latency and correctness measurements from the 2026-09-07 review. **No per-feature Elo ablation (feature on/off, games, interval) has been run**, so what each technique is worth in Elo on this engine is not known; the evidence here is that each does what it is supposed to do, not what it scores.

technique	what it does and why	measured effect (local, dev box)
Iterative deepening with soft and hard deadlines	No new iteration after the soft deadline; abort mid-iteration at the hard deadline and return the last completed iteration’s move (or the current one’s if a later root move already beat the first).	Depth 5–6 at 120 s + 0.5 s, 4–6 at 20 s + 0.2 s (source: RESULTS.md, “depth” rows).
Check extension	In check, search one ply deeper (capped at <code>MAX_PLY = 128</code>), applied before the TT probe so probe and store see the same depth (review item L: the probe had accepted entries one ply too shallow).	Correctness fix; pinned by a unit test in <code>tests/test_search.py</code> (check extension before the table probe).
Quiescence, stand pat first	Test the static evaluation against β before generating any capture; a position with no legal move is still scored as mate or stalemate, never evaluated. Before the change, captures were generated and then discarded at seven quiescence nodes in ten.	+43 % nps in isolation (reviewer); all review fixes together 51.7k $\rightarrow$ 67.2k nps, identical results (source: DECISIONS.md item K; RESULTS.md, “review nps”).
Evasion cap, <code>QS_EVASION_PLIES = 4</code>	In check inside quiescence, every evasion is searched for the first four quiescence plies; deeper checks stand pat like any other node so dense positions cannot explode.	Eight queens a side: one depth-1 iteration fell from hundreds of thousands of nodes to 33k (source: DECISIONS.md item M).
Transposition table, int-only entries	<code>dict</code> keyed by position, entry (<code>depth</code> , <code>score</code> , <code>flag</code> , <code>move_code</code>) with every field an <code>int</code> (the move is packed as from to $\ll$ 6 promotion $\ll$ 12 and rebuilt at the probe). Mate scores are stored ply-independent. Cap 250 000 entries; emptied at move start if above 60 % of the cap, and as a backstop when the cap is hit mid-search. Persists across moves within a game.	Worst generation-2 garbage-collection pause in a 60 s search: 143 ms (270k <code>chess.Move</code> entries) $\rightarrow$ 13.7 ms; RSS at the cap 143 MB $\rightarrow$ 120 MB; nps on that position 50.3k $\rightarrow$ 62.7k (source: RESULTS.md, “gen-2 gc pause” and “RSS at the TT cap”). A 143 ms stall is invisible to the clock check and would have been about three times longer on the platform’s core.
Clock check every <code>NODE_CHECK_INTERVAL</code> nodes (128 here; 512 in the compiled engine , which is the one that plays — at 505 k nodes/s that is the same millisecond of granularity, and equal node counts would have meant unequal time)	Read <code>perf_counter()</code> and abort past the hard deadline or past an optional node limit.	With 1024 the search overshot the hard deadline by 25 ms mean, 65 ms worst locally (9 ms worst in the integration game); a clock read costs about 60 ns, and nps is unchanged at 128 (source: DECISIONS.md item A; RESULTS.md, “hard-deadline overshoot”).

technique (continued)	what it does and why	measured effect (local, dev box)
Move ordering	TT move first; captures and promotions by MVV-LVA (victim value $\times 10$ minus attacker value; a promotion counts the promoted piece as the victim); two killer moves per ply (quiet moves that caused a cutoff at the same ply); remaining quiet moves by the history heuristic (bonus depth ² per cutoff, indexed by colour, from- and to-square). Captures are never stored as killers.	Structural; no separate measurement.
Repetition inside the tree	A node whose position already occurred in the game history or on the current search path scores 0. The side that is ahead therefore avoids repeating and the side behind seeks it, with no root rule (source: DECISIONS.md, “Two-fold repetition”).	Pinned by <code>test_side_ahead_avoids_repetition</code> and <code>test_side_behind_seeks_repetition</code> .
Graph-history mitigation	A draw found through the path is a fact about the line, not the position, so a node whose value depended on one is not stored with its score; only its move is kept as an ordering hint at depth -1 , never over an entry earned without the repetition. Draws from the game history are permanent within the game and are stored normally.	Cost 0.15 % of nps (A/B, three 3 s searches each). Evidence: after a depth-7 search of 8/8/8/4k3/8/8/8/R3K3 w the table held a depth-5 lower bound of 0 for a position whose fresh value is -628 (source: DECISIONS.md item E).
Root tie-break, <code>DRAW_TIEBREAK_MARGIN</code> = 300	When the best root score is exactly 0, at least two root moves tie at it and the root static evaluation is at least 300 in our favour, the tied move whose child evaluates best is chosen (stalemates excluded): a rule draw inside the horizon must not stall a mop-up.	With the urgency clamps (Section 7) the 600-ply case 8/8/8/3K4/8/8/8/q6k b - - 0 291 converts (mate in 17 plies); it was drawn at the cap before (source: RESULTS.md, “Known unconverted wins”).
Queen over under-promotion on exact ties	If the first root move is an under-promotion, the queen promotion of the same pawn is searched before it, so the queen wins an exact tie (fail-soft scores of later moves are only bounds, so comparing them at the root was rejected).	The reviewer had seen <code>b8=R</code> chosen over <code>b8=Q</code> (source: DECISIONS.md item H).
Urgency clamps	Not a search technique but a budget one: when the 600-ply cap or, in a pawnless mop-up, the fifty-move draw is near, <code>moves_to_go</code> is clamped to the moves left before it so the search gets the time a mate needs (Section 7).	Two won KQ-vs-K games had been drawn by rule with the normal budget (source: DECISIONS.md item C).

6 Evaluation

Terms. The static evaluation (`mikhail_letal/evaluation.py`) has three terms: material, piece-square tables and a mop-up term. It is *tapered*: every piece has a middlegame table and an endgame table, and the position’s *phase* blends them. The phase counts the non-pawn pieces

on the board with weights knight 1, bishop 1, rook 2, queen 4, so it is 24 with every piece on the board and 0 with only kings and pawns; the score is

$$\text{score} = \frac{\text{mg} \cdot \text{phase} + \text{eg} \cdot (24 - \text{phase})}{24},$$

truncated toward zero so that mirrored positions score exactly opposite. Material values are folded into the tables at load time, so the hot loop does one table lookup per piece per phase. Insufficient material scores 0. The result is negated for Black to move, so the search sees every node the same way. Piece bitboards are read directly rather than through `pieces_mask` and `scan_forward` (same semantics, 1.7× faster) (source: INTEGRATION_NOTES.md, “Builder deviations”).

Piece values. Pawn 100, knight 320, bishop 330, rook 500, queen 900, in both phases. These are the universal textbook values, recorded as such and untuned (source: weights/PROVENANCE.json).

The parametric prior. Each of the twelve tables (six pieces × two phases) is computed by `tools/gen_pst.py` from the square’s geometry. Two helper quantities: *centrality* is $3 - \max(|\text{file} - 3.5|, |\text{rank} - 3.5|)$, which is 2.5 on the four centre squares and −0.5 on the rim; *advancement* is the rank seen from the piece’s own side, 0 to 7. Then: pawns gain `pawn_adv_mg` per step forward and `pawn_centre_mg` on d/e files on ranks 3–5 in the middlegame, and `pawn_adv_eg`· $\text{advancement}^2/7$ in the endgame (a passed-pawn shape); knights gain `knight_centre` × centrality and lose `knight_rim` on the outer ring; bishops gain `bishop_centre` × centrality and `bishop_long_diagonal` on the two long diagonals; rooks gain `rook_seventh` on the seventh rank and `rook_centre_file` on d/e files in the middlegame and are flat in the endgame; queens gain `queen_centre` × centrality × 0.5 and lose `queen_early` for leaving the back two ranks in the middlegame; the king gains `king_shelter` on b1/c1/g1 and loses `king_centre_penalty` per rank of advancement and on the d/e files in the middlegame, and gains `king_centre_eg` × centrality in the endgame. The script prints every table as an 8×8 grid for a human to read, records the parameters and the tables’ SHA-256 in the JSON’s `_provenance` object, and a test checks that the shipped file matches the generator bit for bit (`test_pst_json_matches_the_generator`).

parameter	value	reason recorded in <code>tools/gen_pst.py</code>
<code>pawn_adv_mg</code>	4	each step forward gains space, but pawn storms weaken the king, so small
<code>pawn_centre_mg</code>	12	d/e pawns on ranks 3–5 hold the centre; comparable to a tempo
<code>pawn_adv_eg</code>	14	quadratic in steps forward ($14 \times 25/7 = 50$ on the 7th), a passed-pawn shape
<code>knight_centre</code>	12	knights gain most from the centre ($2.5 \times 12 = 30$) and lose most on the rim
<code>knight_rim</code>	14	a knight on the edge attacks half as many squares: extra penalty
<code>bishop_centre</code>	6	bishops act at range, so centrality matters half as much as for knights
<code>bishop_long_diagonal</code>	6	the two long diagonals see the whole board
<code>rook_seventh</code>	20	a rook on the 7th cuts off the king and eats pawns
<code>rook_centre_file</code>	8	d/e files are open most often in the middlegame
<code>queen_centre</code>	8	halved by the formula: the queen is strong everywhere
<code>queen_early</code>	12	a queen past the second rank invites harassment before the minors develop
<code>king_shelter</code>	25	castled squares b1/c1/g1 keep pawns in front of the king
<code>king_centre_penalty</code>	10	per rank of advancement and for the d/e files, so e8 gives -80 at most
<code>king_centre_eg</code>	10	the king becomes a fighting piece; $2.5 \times 10 = 25$ in the centre
<code>mopup_edge</code>	10	per Manhattan step of the bare king from the centre (max 6, so 60)
<code>mopup_close</code>	12	per step our king is closer than 14; exceeds one ring of <code>king_centre_eg</code>

Table 3: All sixteen parameters of the prior (`PARAMETERS` in `tools/gen_pst.py`, copied into `weights/pst.json` under `_provenance.parameters`). Magnitudes were chosen so the largest table entry stays under a pawn: piece-square effects refine material, they do not rival it. Resulting table ranges (min, max): middlegame P 0..24, N $-20..30$, B $-3..21$, R 0..28, Q $-14..2$, K $-80..25$; endgame P 0..50, N $-20..30$, B $-3..21$, R 0..0, Q $-2..10$, K $-5..25$ (source: `weights/PROVENANCE.json`).

Mop-up. When one side has a bare king, the other has no pawns and at least a rook’s worth of material (or two minor pieces), the stronger side gains `mopup_edge` · $d_{\text{centre}}(\text{weak king}) + \text{mopup_close} \cdot (14 - d(\text{king, king}))$, with d the Manhattan distance: drive the bare king to the edge and bring ours close. The idea is textbook geometry; the two weights (10 and 12) are ours and were chosen by hand so that “close” beats one ring of the endgame king table (source: `DECISIONS.md`, 2026-09-06, “Mop-up term”).

Cost. `evaluate()` takes 3.4–3.8 μs in the middlegame and 1.6 μs in the endgame (10 000-calls loops) (source: `RESULTS.md`, row “`evaluate()` cost”).

How the weights were obtained: tuning was tried and rejected. The shipped weights are the parametric prior as generated, untuned. That is not because tuning was skipped — it was run, and it lost.

Texel-style tuning of the piece values and tables was implemented in `tools/tune_texel.py` and measured against the untuned set under the ordinary promotion rule. The first attempt scored 36.0% over 300 games, an interval of -100 Elo (-140 to -62), comfortably *below* zero. A second attempt with stronger regularisation recovered most of the loss but still did not clear the bar: 46.7%, -23 Elo (-61 to $+14$) (source: `RESULTS.md`, rows `v0.3-texel-vs-v0.2-10s` and `v0.3-texel-lambda40-vs-v0.2-10s`). Under a rule that promotes only on an interval above zero, the untuned prior stands.

Both runs are kept in `RESULTS.md` rather than deleted. The most likely reading is over-fitting: the tuning set is small next to the number of free parameters in twelve 64-entry tables, and a prior built from a geometric formula generalises better than weights fitted to a few thousand positions. It is recorded here as a negative result because a report that lists only what worked is not evidence of a method.

7 Time management

The budget formula. Every constant lives in one frozen dataclass, `TimeParams` (file `mikhail_letal/timing.py`), so that a calibration changes one object and nothing else. With t the clock in milliseconds before the move and m the number of moves we have played this game:

```
moves_to_go = clamp(50 -  $\lfloor 0.7m \rfloor$ , 20, 50),
moves_to_go = min(moves_to_go,  $\lceil \text{plies\_to\_cap}/2 \rceil$ ,  $\lceil \text{fifty\_move\_room}/2 \rceil$ ),
    soft =  $(t - \text{overhead})/\text{moves\_to\_go} + 0.8 \times 500$ ,
    hard = min( $3 \times \text{soft}$ ,  $0.25t$ ),
    floor = max(1500,  $0.05t$ ),
    hard = min(hard,  $t - \text{overhead} - \text{floor}$ ),    soft = min(soft, hard),
```

both clamped to be non-negative. `plies_to_cap` = 600 – ply is always passed; `fifty_move_room` = 100 – halfmove clock is passed only in a pawnless mop-up, where nothing but the mate itself resets the halfmove clock (the urgency clamps of review item C). Worked examples from the unit tests: at $t = 120\,000$, $m = 0$: `moves_to_go` 50, `soft` = $119\,950/50 + 400 = 2799$ ms, `hard` = 8397 ms; at $t = 60\,000$, $m = 20$: `moves_to_go` 36, `soft` = $59\,950/36 + 400 \approx 2065$ ms, `hard` ≈ 6196 ms (source: `tests/test_timing.py`).

Where the divisor comes from. 50, 20 and 0.7 are not chosen by feel. 1 697 finished ladder games were collected and used to measure how many of *our* moves are really left at each point of a game, given the game reached that point: the median is 67 at the start, falls by about one a move to roughly 25 by move 50, and then flattens. If a move spent its whole soft budget, sharing the clock over that curve would be the right answer directly. It does not — a move spends about 0.70 of its soft budget — so the divisor is that curve scaled by 0.70, which makes the *realised* spending an even split of the clock over the moves that are really left. Hence $0.7 \times 67 \approx 47 \rightarrow 50$ at the start, and a decay of 0.7 divisor-moves per move played.

The floor is the interesting one, and it is set by a safety argument rather than by the median. At full spend the clock settles where a move’s cost equals the increment, at `overhead + moves_to_go_min  $\times$  500  $\times$  (1/usage – 0.8)`. At a floor of 16 that lands on 1650 ms — *exactly* `panic_ms`, the threshold below which the engine is skipped entirely, so the tail of a long game would settle precisely on the boundary where it stops searching. At 20 it is 2050 ms at full spend and about 6.3 s at the measured 0.70, and over the ladder games that ran to at least 90 of our moves the lowest clock reached rises from 4.6 s to 5.9 s. The floor is therefore 20.

How the budget is used. `get_move` starts its own timer on entry, so board construction and history bookkeeping count against the budget as they do on the platform. The searcher is handed two absolute deadlines: the soft budget as a target, and a hard deadline at which the search aborts mid-iteration, with the clock read every 512 nodes in the compiled search (Section 5). Below `panic_ms` = 1650 ms the engine is skipped and the fallback plays (Section 9); a position with a single legal move is played without searching.

Starting the next depth: a prediction, not a fixed share. Whether to begin depth $d + 1$ used to be decided by a fixed fraction of the budget — start one only before $0.45 \times \text{soft}$ had elapsed. The platform’s own logs showed what that does. Per-move times came in two lumps and nothing between: 1.5–3.0 s, where an iteration finished and the engine stopped with a third of its budget unspent, or 8.9–10.1 s, where it started a depth that could not finish and ran into the hard ceiling. The cause is arithmetic. Iteration costs grow geometrically — each depth costs

about 4–5 times the previous — so a depth begun at 0.45 of the budget cannot finish inside it, and a fixed fraction has no way to tell the two cases apart.

The rule now predicts. The cost of depth $d + 1$ is estimated as $\rho \times T(d)$, where ρ is measured from the last two completed iterations, clamped to $[2.0, 8.0]$ and defaulting to 4.5 before there is data; the depth is started only if it is predicted to end inside a target. The target is the soft budget times `iteration_target_factor`, stretched by half again when the best root move changed at the last completed depth, and cut to 0.7 when it has been stable for six iterations with a score that is not falling. Insisting that the next depth *finish* by the soft budget is the obvious reading and the wrong one: with geometric costs it stops a factor of ρ short and leaves a third of the clock unspent, which is why the target factor is 1.35 rather than 1.0.

Every target is clamped to the hard window, so the guarantee that matters is preserved by construction: a decision to start an iteration implies the elapsed time is still inside the hard deadline. The abort path and the flag invariant are untouched, and `tests/test_timing.py` asserts the guarantee over a grid of histories and windows. The old fixed fraction survives only as a fallback for an iteration too short to measure (under 1 ms), where a ratio would be meaningless.

constant	value	origin
increment_ms	500	agent contract
overhead_ms	50	measured: the platform charged 0–2 ms beyond what we measure ourselves (mean 1.1) over 25 rated moves; this file’s rule is max +50
moves_to_go_max / min / decay	50 / 20 / 0.7	measured: 1 697 finished ladder games, scaled by the 0.70 of its budget a move actually spends; the floor is set by the settling-clock argument above
increment_fraction	0.8	brief §6.2
hard_multiplier / hard_fraction	3.0 / 0.25	brief §6.2
floor_ms / floor_fraction	1500 / 0.05	brief §6.2
panic_ms	1650	below it the engine is skipped and the fallback plays. Deliberately unchanged when overhead_ms fell, so that nothing within a second and a half of the flag behaves differently from the build that was measured
next_iteration_fraction	0.45	brief; demoted to a fallback for an iteration shorter than ratio_measurable_s = 1 ms, which says nothing about the next one
iteration_ratio_default / min / max	4.5 / 2.0 / 8.0	measured: the platform’s per-move logs give 4–5× per depth; clamped so one mis-timed iteration can neither stop the search early nor start one it cannot finish
iteration_target_factor	1.35	measured: 1.0 spends 0.70 of the budget for mean depth 10.83; 1.35 spends 0.80 for 11.17; 1.75 reaches the hard ceiling
unstable_factor	1.5	hand-chosen, textbook magnitude: half a budget more when the best root move just changed. Bounded by the hard ceiling like every other target
easy_factor / stable_depths / score_drop	0.7 / 6 / 30 cp	measured against 0.5 / 4 / 30, which fired on nearly every move and spent less than the fixed rule it replaced
cold_finish_fraction	0.25	the share of the first move’s clock that may go on finishing a warm-up that ran out of its own budget. v1.0 only: the bound it expresses was found not to hold (Section 3) and the constant is deleted in the successor, where the cold finish is unbounded and its cost is charged to the move’s clock instead

Table 4: Time-management constants of build v1.0, calibrated on 8 September against the platform’s own per-move timing and 1 697 ladder games (source: docs/CALIBRATION.md; docs/PROVENANCE.md). The worst case an iteration can be *started* for is $1.35 \times 1.5 = 2.03$ soft, against a hard ceiling of 3.0 soft, so the abort path is no likelier to be needed than under the fixed rule.

What the local games show. At the real clock the smoke games from the extracted zip ran 1.5 to 7.7 s per move against a soft target of about 3.3 s and a hard limit of about 10 s, with an average spend of 3.1–3.5 s per move including the increment (source: RESULTS.md, row “depth at 120 s clock”; INTEGRATION_NOTES.md, “Time profile”). The reason moves overrun the soft target is the iteration-cost ratio $T(d)/T(d-1)$: median 4.3–5.2 and up to about 10 on middlegame searches, so an iteration begun at $0.45 \times \text{soft}$ usually runs to $2\text{--}2.5 \times \text{soft}$ and is cut by the hard limit; below about 20 s on the clock the hard limit ($0.25t$) binds every move

(source: CALIBRATION.md, “Observations”). This is sustainable at 40 moves to go but is the first thing to check in the validation log. Worst hard-deadline overshoot inside `get_move`: 9 ms (`t 2661` against `h 2652`), with the 1024-node check interval then in force; the local harness’s own overhead (referee-measured minus self-measured) was 0–1 ms per move (source: RESULTS.md, rows “hard-deadline overshoot” and “local harness overhead”).

Calibration procedure. When a validation log arrives the operator pastes it and the following is done (source: CALIBRATION.md, “Procedure”): (1) extract the init time, every per-move time, the clock left after each move and any stderr; (2) pair each platform-measured move time with the self-measured `t` token of our log line; the difference is the platform overhead for that move, and `overhead_ms = max(difference) + 50`; (3) compute the platform node rate from our `nps` tokens and divide by the local node rate on the same build and positions to get the speed factor, and rescale any node cap; (4) record the platform init time against the local one; (5) commit the new constants with the log excerpt. **That procedure has been run.** The interpreted v0.2.1’s rated logs of 8 September gave the 2.3x speed factor and the 0–2 ms per-move overhead that set `overhead_ms`; v1.0’s own validation and its rated rounds gave the start-up figures and the depth and node rates quoted in the v1.0 addendum; and round 74 supplied a full per-move table for a whole rated game (source: CALIBRATION.md, “Log of calibrations”; handoff/LOG-round-74-zagreus.md). Every time constant in Section 7 is calibrated against those measurements rather than against the brief’s initial values.

The one part of the procedure that has **not** converged is start-up. Six samples of the compiled build’s import range over 33.2–49.4 s of numba compilation against a 90 s budget, and the five fast ones are all uncontended single rated games while the slow one is half of a concurrent validation pair — so the samples do not yet bound the case where two of our games compile at once, which the agent contract permits. That is recorded as the build’s largest open risk in the addendum rather than resolved here.

8 Endgame handling

Mop-up. Section 6: a geometric term so that KX-vs-K converts without tablebases.

Tablebases. Not adopted in this build. Syzygy 3–4-man probing through `chess.syzygy` is a Stage 2 experiment; it ships only if the files fit inside the size cap with margin and an arena run from endgame-heavy positions shows a positive interval, otherwise the mop-up term stays (source: DECISIONS.md, 2026-09-06, “Mop-up term instead of tablebases”).

Draw handling. Inside the tree the terminal checks run in this order: `ply ≥ 600` is a draw; a position already in the game history or on the search path is a draw; `halfmove clock ≥ 100` is checkmate if the side to move is mated, otherwise a draw (the referee checks mate first, and so do we: `test_mate_in_one_beats_the_fifty_move_rule`). The game history starts at the first FEN received, as the referee’s does, and is kept by `GameState` (Section 9). Because any earlier occurrence scores a draw, the engine steers away from a repetition before the third occurrence when ahead, and toward one when behind. Near a rule deadline the urgency clamps (Section 7) and the root tie-break (Section 5) make sure a won position is not shuffled into a draw.

The 600-ply cap. `GAME_PLY_CAP = 600`; the ply is taken from the FEN’s move number, so the count matches the referee’s, which counts the opening position (source: tests: `test_ply_cap_scores_draw`).

position	outcome at 5 s/move	note
Lucena 1K1k4/1P6/8/8/8/8/r7/2R5 w - - 0 1	not converted	rook-endgame technique beyond a depth-6 horizon; Stage 2
8/8/8/8/3k4/8/8/4KQ2 w - - 82 60 (KQ vs K, 18 plies of fifty-move room)	not converted: draw by fifty moves, mate two moves short	the tablebase (queried by hand) gives mate in 13 plies, so it is winnable; the engine drives the king to the edge but not at tablebase pace, and depth 6 at 1 250 ms per move never sees the mate in time
8/8/8/3K4/8/8/8/q6k b - - 0 291 (KQ vs K, 19 plies before the 600-ply cap)	converted, mate in 9 moves (17 plies)	drawn at the cap before the 2026-09-07 fixes

Table 5: Endgame probes with the reviewer’s driver at 5 000 ms per move, where the hard limit ($0.25 \times$ clock) caps every move at 1 250 ms; the opponent is a second copy of the engine (source: RESULTS.md, “Known unconverted wins”). At the real 120 s clock the urgency clamps can add time; at 5 s they cannot.

9 Safety and failure handling

The wrapper. `agent.py` is deliberately small. `get_move` builds the board from the FEN, updates the game history, lists the legal moves, decides a budget, calls the engine, and checks the answer. The whole body is inside one `try`: any exception is logged in one line and the fallback plays. The engine’s move is accepted only if it is in the legal-move list; then a *final guard* (`_validated`) parses the UCI string back and looks it up in the legal moves of a fresh `chess.Board(fen)`, exactly as the referee will, replacing it with the fallback if that fails. Only after this check is the move recorded in the history and printed. Even the logging is wrapped, and lines are capped at 120 bytes because the platform keeps only the first and last 4 KB of output.

The fallback. `fallback_move` looks one ply ahead: a mate in one if there is one, otherwise the move that leaves us the most material, preferring big captures and queen promotions, with the UCI string as the last tie-break so the same position always gives the same move. It returns in a few milliseconds and is tested on its own (mate in one, best capture, promotion, en passant, determinism, legality in random positions, speed) (source: tests/test_fallback.py).

The panic band. Below `panic_ms = 1650` ms the engine is not called at all. The threshold equals `overhead_ms + floor_ms`: in the band 1500–1650 ms the formula returned a zero budget and the search aborted at its first clock check with a depth-0 move, so the band was routed to the fallback instead (source: INTEGRATION_NOTES.md, “For the reviewers” item 2; commit 974264d).

Desync reset. The platform sends nothing but a FEN and a clock. `GameState.observe` expects each received position to be one legal move away from the position after our last move, and reconstructs the opponent’s move by trying every legal move and comparing repetition keys. If no move matches (a desync: the platform and our history disagree), the history is reset to the received position, a line is logged, and the engine’s transposition table is cleared, because stored draws may rest on the history just discarded (source: `agent.py`; DECISIONS.md item E). One known gap is benign by design: when we play Black we never see the game’s true start

position, so its count stays one below the referee’s; since any earlier occurrence already scores a draw, nothing changes (`test_black_never_sees_the_start_position`).

Start-up. A depth-2 search from the initial position runs at import so every code path has executed before the clock starts; the table is cleared after it. `OMP_NUM_THREADS` and `NUMBA_NUM_THREADS` are set to 1 before any other import. Log lines are printed with `flush=True`: the runner points our stdout at a pipe, which Python block-buffers, and kills the process when the game ends; without the flush the first local game printed no agent output at all (source: `INTEGRATION_NOTES.md`, “What changed in integration”).

Fuzz counts. 300 games against `baselines/random` at 3 s + 0.05 s and 200 against `baselines/greedy` at 10 s + 0.1 s, over the collected openings, twelve games in parallel: 500 games, 500 wins by checkmate, zero terminations by crash, illegal move, flag or init (source: `RESULTS.md`, rows `fuzz-random-v0.1` and `fuzz-greedy-v0.1`). The random mover is the opponent that plays the strange moves (under-promotions, en passant, stalemate tries) that exercise the guard paths.

The compiled engine was fuzzed the same way: 200 games against `baselines/random`, 200 wins by checkmate, zero failure terminations (source: `RESULTS.md`, row `numba-fuzz-random`). A re-run on the final build was started and **stopped at 25 games of 200** to give the machine to a measured match, and is recorded as stopped rather than quietly dropped: 25 games, 25 wins by checkmate, no failed termination, lowest clock 1 649 ms of 3 000. The final build’s legality argument therefore rests on the full test suite — which includes a legal move being required from the compiled search on 3 000 sampled positions at three node limits, and the property suite driving `agent.get_move` — rather than on a completed 200-game fuzz. That is a real gap and it is stated as one: a gate that was not finished should not be read as one that was.

10 Testing methodology and results

Machines and load. Development machine: 16 cores, 7.8 GB RAM, WSL2 on Windows, Python 3.12.14. Platform: one core of an AMD EPYC 9V74 at 2.60 GHz, 2 GB (source: `RESULTS.md`, “Machine”). Every arena row records the one-minute load average at the start and the end of the run, because games sharing a machine stop measuring time management; the deciding games (real clock, previous version, strongest baseline, ≥ 40 games with the slowest move and minimum clock inspected) must be run alone.

Statistics. `tools/arena_openings.py` draws openings from `data/openings.txt` (219 distinct FENs under 34 names, collected from 400 public game pages of 80 of the 335 teams, one request per second, on 6 September) (source: `RESULTS.md`, row “curated openings collected”), alternates colours, and prints the same score, 95 % interval and Elo formulas as the starter harness, plus draw rate and termination counts. Rule of thumb for sample size (score near 50 %, about 25 % draws): the Elo standard error is about $300/\sqrt{n}$, so about 300 games resolve +50 Elo and about 1 500 resolve +20; no improvement whose interval includes zero is ever claimed (source: brief, §7.3).

opponent	tc	games	+W =D -L	score	95 %	terminations	workers	load start / end
random	3 + 0.05 s	300	+300 =0 -0	100.0 %	±0.0 %	checkmate 300	12	0.10 0.27 0.22 / 7.45 2.31 0.93
greedy	10 + 0.1 s	200	+200 =0 -0	100.0 %	±0.0 %	checkmate 200	12	7.45 2.31 0.93 / 10.55 5.77 2.43
minimax	120 + 0.5 s	40	+39 =1 -0	98.8 %	±2.4 %	checkmate 39, threefold repetition 1	4	1.48 3.05 2.04 / 2.66 3.91 3.34

Table 6: Arena runs of build v0.1.0: rows **fuzz-random-v0.1**, **fuzz-greedy-v0.1** and **v0.1-vs-minimax-real-clock** of RESULTS.md, in that order; opponents are the starter’s **baselines/random**, **greedy** and **minimax**. Openings: **data/openings.txt**. The tool printed no Elo for these runs. Loads of 7–10 during the fuzz runs are expected with twelve workers and mean those games do not measure time management; the minimax run used four workers (load 1.5–3.9 on 16 cores), so its per-move times are indicative but it is not the solo pass of Stage 3. Its one draw is by threefold repetition; the PGNs are kept under **data/pgn/**. **baselines/greedy** searches one ply on material; **baselines/minimax** two plies on material and mobility with no time management (source: README.md).

measurement	value	how
node rate, middlegame	50–61k nps	agent log lines, harness.play vs greedy at 20 s + 0.2 s
depth at 20 s clock	4–6 (seldepth 13–21)	same game
depth at 120 s clock	5–6 (seldepth 18–32); 1.5–7.7 s per move vs soft $\approx$ 3.3 s, hard $\approx$ 10 s	harness.package smoke games from the extracted zip
hard-deadline overshoot inside get_move	9 ms worst seen	clock read every 1024 nodes at the time
local harness overhead	0–1 ms per move	referee clocks minus the t token
import time (incl. depth-2 warm-up)	0.033 s	three runs
peak RSS after a 10 s search	42.4 MB (16.8 MB before); 53 640 TT entries	resource.getrusage
evaluate() cost	3.4–3.8 μ s middlegame, 1.6 μ s endgame	10 000-call loops
zip	23 333 bytes, 66 825 unzipped	harness.package ; both smoke games clean
test suite	143 passed (44 property tests ran)	pytest -q , 33 s
review nps (3 s search, start position)	51.7k $\rightarrow$ 67.2k (+30 %), same move, depth 6/17	median of 3
gen-2 gc pause, 60 s search	143 ms max $\rightarrow$ 13.7 ms max	gc.callbacks timing
RSS at the TT cap	120 MB (250k entries) after; 143 MB (270k) before	same 60 s search

Table 7: Other measurements of build v0.1.0, development machine (source: RESULTS.md, “Other measurements”).

Static checks and smoke games. At the Stage 0 integration pass: **ruff format -check**: 32 files already formatted; **ruff check .**: all checks passed; **mypy -strict**: no issues in 28 source files; **pytest -q**: 143 passed in 32.8 s; **harness.arena** vs random, 2 games at 5 s: +2 =0 -0 by checkmate; **harness.play** vs greedy at 20 s + 0.2 s: won by checkmate in 16 of our moves, twice; **harness.package**: zip built and both platform-style smoke games (120 s + 0.5 s, 20 plies) clean (source: INTEGRATION_NOTES.md, “Checks run”).

On the shipped v1.0, measured 8 September: **ruff check** and **ruff format -check** clean;

mypy in strict mode over `agent.py`, `harness`, `mikhail_letal`, `tools` and `tests`: no issues in 50 source files; `pytest -q`: **621 passed** in 2 min 41 s, with the gates at their reduced sizes. The suite has more than quadrupled since Stage 0, which is where most of the compiled engine’s correctness argument lives: the `perft`, `legal-move-set`, `make/unmake` and `evaluation-parity` gates above are all part of that count.

Perft and the compiled board’s gates. Stage 0 needed no `perft`, because `python-chess` generated the moves. Stage 1 has its own move generator, so it needs one, and `tests/test_fastboard.py` is the gate. **The `perft` counts are not taken from a published table.** `chess_perft` walks `python-chess`’s own legal move generator to produce the expected number, so a disagreement is a disagreement with a reference implementation rather than with a number copied from a web page — which also means the gate cannot be passed by having memorised the standard answers. Four gates run, with `python-chess` as the oracle throughout, and `NUMBA_BOUNDSCHECK=1` making `numba` check every compiled array index:

1. **Perft** to depth 4 on the standard public positions and on 58 positions built to break one rule each, and to depth 3 on all 219 curated openings and on 2 000 positions from random playouts.
2. **Legal-move-set equality**, compared as sets of UCI strings, on 100 000 positions, with the sample audited to confirm it contains en passant, all sixteen castling-rights combinations, check, double check, promotions, pawns on the seventh rank and terminal positions — so that the count cannot be met by easy positions alone.
3. **Make/unmake round trip**: after every make and every unmake in a Python-level `perft`, the FEN including both clocks is byte-identical, the metadata is unchanged, and `check_invariants` passes.
4. **No jitted function gains a new signature under load**, so nothing compiles after import — the property Section 3 explains the warm-up exists to guarantee.

The evaluation has the equivalent gate against `evaluation.py` described in the v1.0 addendum. The sizes above are the full ones, run with `LETAL_FULL_GATES=1` before a promotion; an ordinary `pytest` run uses reduced samples so that the suite stays quick.

Per-move log of the minimax run. The arena tool cannot see the agent’s log, so three further games were played alone on the machine with `harness.play` (two as White, one as Black, all won by checkmate): 71 engine moves, one of them 2 ms over its hard cap (the clock is read every 128 nodes), slowest move 8 429 ms against a cap of about 10 s, minimum clock before a move 42 958 ms, mean 2 526 ms per move against a mean soft budget of 2 925 ms, depth 1–7 (median 5), median 64 270 nps, import 31–34 ms (source: `RESULTS.md`, “v0.1 solo real-clock pass”). The one repetition draw of the 40-game run was a position where the engine was a pawn down (material 20 vs 21) and steered into the repetition, which is the intended behaviour when behind.

Final measurement pass. [PENDING: Stage 3 final pass alone on the machine at 120 s + 0.5 s: vs the previous version, vs the strongest baseline, and ≥ 40 games with the slowest move and the minimum clock reached].

11 Rating estimate

This section is a method statement, and it now describes two builds. **The rating below belongs to the interpreted v0.2, not to the shipped v1.0.**

v0.2 (interpreted), fully measured. 300 real-clock games against Stockfish 19 at UCI_Elo 1600–2400 on 8 September (60 per level, six games at a time; scores 70.0, 76.7, 45.0, 38.3 and 43.3 %; zero flags): CCRL-40/4-scale ≈ 2050 (95 % interval 1940–2170: bootstrap 2000–2110 plus ± 100 yardstick calibration in quadrature); FIDE-equivalent ≈ 1840 –2360 (Route A: CCRL–100 as the lower bound, TalkChess 2800 – $0.7 \times (2800 - \text{CCRL})$ as the upper); chess.com Rapid-equivalent ≈ 1820 –2340; chess.com Blitz-equivalent ≈ 1730 –2700 (ChessGoals table, interpolated). Route B (the Sunfish bot) was not run.

v1.0 (compiled), provisional and under-sampled. The same instrument and the same fit, run on 64 games at UCI_Elo 1800–2400 (16 per level; scores 84.4, 75.0, 65.6 and 56.2 %; zero flags) (source: RESULTS.md, rows v1.0-vs-sf*-real), give a maximum-likelihood CCRL-40/4-scale rating of ≈ 2305 , bootstrap 95 % interval 2199–2426, widening to 2159–2462 once the yardstick’s own ± 100 calibration uncertainty is added in quadrature.

This is quoted as provisional and should not be repeated as the build’s rating. The protocol in this section calls for at least 60 games per level; 16 is barely a quarter of that, the sample is one fifth of what produced the v0.2 figure, and the levels stop at 2400 while the build scores above 50 % there — which is exactly the condition under which the protocol says to extend upward, because a fit whose top level is not a losing one is extrapolating at its own boundary. The human-scale conversions are deliberately not computed from it. **[PENDING: full-sample re-rating of v1.0: ≥ 60 games per level, levels extended above 2400 until the build scores under 50 %]**

Why the ladder’s anchors cannot be mapped to CCRL directly. Five house engines hold fixed ladder ratings and also have published CCRL Blitz (40/4) ratings. The two sets do not agree even in order.

house engine	CCRL Blitz 40/4 (fetched 6 Sep 2026)	ladder anchor
Loki 3.0.0	2427 ± 13	1721
Zagreus 5.0	2168 ± 16	1686
Rustic Alpha 3.0.0	1791 ± 16	1630
Shallow Blue 2.0.0	1576 ± 19	1632
Zagreus 2.3.1	1399 ± 20	1527
spread	1 028	194

Table 8: The five fixed-rating house engines (source: brief, §8.1).

A 1 028-point CCRL spread compressed to 194 ladder points, not in CCRL order, with similar win-loss records, means the organisers are almost certainly running these engines throttled (fixed depth or nodes or similar) and set the anchor values by their own calibration. We therefore do not derive a ladder-to-CCRL mapping from the anchors; both sets of numbers are recorded here and that is all.

Yardstick matches (local only). The instrument is tools/yardstick/agent.py, a harness-compatible agent that drives a UCI engine through chess.engine.SimpleEngine on the real clock it is handed, with Threads=1, Hash=16, Move Overhead=100, UCI_LimitStrength=True and UCI_Elo at a level taken from an environment variable. The engine is a locally installed Stockfish, whose UCI_Elo is documented as calibrated at 120 s + 1 s and anchored to CCRL 40/4, close to the competition clock; the same wrapper without strength options runs the real Sunfish as a second yardstick. Levels 1400, 1600, 1800, 2000 and 2200 (extended upward if the agent scores over 70 % at 2200), at least 60 games per level at 120 s + 0.5 s, colours alternated,

openings from `data/openings.txt`, machine load noted. Fit: the maximum-likelihood rating R over all games with expected score $1/(1 + 10^{(L-R)/400})$ against level L and draws counted as one half; a bootstrap over games for the 95 % interval; then the yardstick’s own calibration uncertainty (± 100) added in quadrature. Reported as “CCRL-40/4-scale $\approx R$ (95 % interval ...)”. Alongside it: the harness’s Elo-vs-baselines numbers, the score against the real Sunfish, and, once the operator has ladder results, the ladder rating, which pins one point of the ladder-to-CCRL map at our own strength.

Conversion to human scales, two independent routes. Engine-pool Elo does not map cleanly to human ratings; a difference of X between two engines is not a difference of X between two humans, and the mapping is not linear, so two routes are used and the union of their ranges is quoted. *Route A (engine pool to FIDE)*: from the CCRL-scale R , the TalkChess rule of thumb $FIDE \approx 2800 - 0.7 \times (2800 - CCRL)$ as the upper end (it is stated to break down below about 1800 CCRL) and $FIDE \approx CCRL - 100$ as the lower end; low confidence. *Route B (human-anchored bot)*: the real Sunfish bot on lichess holds a rapid rating of 1808 against humans, running on PyPy with pondering; on CPython without pondering about 100 lower is assumed, so about 1700 lichess rapid. Our measured Elo difference against Sunfish is compressed by 0.7–1.0 for the engine-to-human transfer and added to that anchor, then converted lichess rapid $\rightarrow$ FIDE and $\rightarrow$ chess.com with the ChessGoals July-2026 comparison table (about 20 000 profiles; its FIDE column has no reliable data below about 1550 FIDE). The chess.com Rapid and Blitz ranges come from the same table by interpolation between these points: FIDE 1740 $\leftrightarrow$ blitz 1500 / rapid 1655; 1865 $\leftrightarrow$ 1800 / 1870; 1965 $\leftrightarrow$ 2000 / 1995; 2075 $\leftrightarrow$ 2200 / 2110; 2190 $\leftrightarrow$ 2400 / 2215; 2300 $\leftrightarrow$ 2600 / 2300. Ranges, never a point.

Final statement. For the **interpreted v0.2**, on 300 games: CCRL-scale ≈ 2050 (1940–2170); FIDE-equivalent ≈ 1840 –2360; chess.com Rapid-equivalent ≈ 1820 –2340; chess.com Blitz-equivalent ≈ 1730 –2700. These are estimates built on (i) a rating-limited reference engine, (ii) one human-rated bot and (iii) a survey-based table; the honest uncertainty is ± 200 or more on the human scales.

For the **shipped v1.0**, no rating is claimed. The provisional 64-game fit above puts it near CCRL-scale 2300, and the direct comparison against v0.2 — +560 Elo over 300 games, an interval far from zero — says the gap is large and well measured. But an engine-pool Elo difference measured at 10 s + 0.1 s against one opponent is not a rating, the yardstick sample behind the 2300 is a quarter of the protocol’s own requirement, and no human-scale conversion is offered from it. The number a judge should be given for this build is the one measured against known opposition: 84.4, 75.0, 65.6 and 56.2 % at UCI_Elo 1800, 2000, 2200 and 2400, sixteen games each, with the intervals shown in the v1.0 addendum.

12 Known limitations and next stage

- **Lucena.** The rook-endgame technique of 1K1k4/1P6/8/8/8/8/r7/2R5 w is beyond a depth-6 horizon; not converted at 5 s per move (source: RESULTS.md, “Known unconverted wins”).
- **KQ vs K at halfmove clock 82.** 8/8/8/8/3k4/8/8/4KQ2 w - - 82 60 is winnable (mate in 13 plies with 18 of room) but is drawn by the fifty-move rule at 5 s per move: the engine makes steady progress but not at tablebase pace, and is two moves from mate when the rule lands. At the real clock the urgency clamps can buy time; at 5 s the hard limit caps every move at 1 250 ms (source: DECISIONS.md item D).
- **Private-API key, narrowed.** `Board._transposition_key()` is private to python-chess and is still called in `search.py`. It is safe at the pinned 1.11.2, and it no longer runs during

a rated game — the compiled engine uses our own Zobrist keys — but the code ships.

Limitations of the shipped v1.0. The Stage 0 list above is kept because it is the history of the build; most of it was closed by the compiled engine. Two of the three unconverted endgames are now won, the speed ceiling is gone, and the speed factor and time constants have been measured against the platform. What remains, in order of how much it is likely to cost:

- **No king-danger term in the evaluation.** The only king-safety term is a count of pawns in front of the king; there is no attacker count, no weighting by attacking piece and no penalty for an open file bearing on our own king. Rated round 70 was lost to exactly this: the engine castled into a queen already bearing on the castled position, and later scored the losing 22...Nxd4 at +305 for itself where Stockfish had the position at -834. Replaying it at up to sixteen times the time it actually spent, and at four to five plies deeper on the compiled engine, changes none of the three losing decisions — so it is not a horizon failure that depth dissolves, but a gap in what the evaluation knows to look for. Analysis by Yan Beletskiy (source: handoff/FINDING-king-safety.md). This was the highest-value evaluation work outstanding and the one whose absence is visible in a lost game rather than in a hunch.
Status. A king-danger term counting attackers into the king’s zone has since been written and is on the development branch, gated on the parity comparison against the Python evaluation like every other term (source: DECISIONS.md, 2026-09-08, “A king-danger term”). It is **not in v1.0**, which is the build this report describes and the build that has played, and it has not yet won a promotion match, so nothing is claimed for it here.
- **Start-up time varies by fifteen seconds, and the sample is two.** The two platform smoke games compiled in 34.8 s and 49.4 s on identical hardware. The budget is 90 s and the warm-up deadline arms at 70 s; a run slower than the worse of the two starts skipping phases and finishes compiling on the first move. See the v1.0 addendum.
- **The evaluation is an untuned prior.** Texel tuning was attempted and *rejected on measurement*: the tuned weights lost 100 Elo over 300 games, and a regularised second attempt still lost 23 with the interval touching zero (source: RESULTS.md, rows v0.3-texel-vs-v0.2-10s and v0.3-texel-lambda40-vs-v0.2-10s). The prior therefore stands, which is an honest negative result rather than an untried idea.
- **One endgame on a knife edge.** 8/8/8/3K4/8/8/8/q6k b - - 0 291 needs 19 of the 19 plies left before the 600-ply cap; it was mated exactly at ply 600 in one replay and drawn in another. Which side of the line a run lands on depends on the wall clock, so it is claimed as neither a fix nor a regression (source: RESULTS.md, “the recorded unconverted wins”).
- **v1.0 has no full-sample rating** (Section 11): 16 games per level against the yardstick, against a protocol that asks for 60, and the top level is one the build still wins.
- **One platform run.** Everything known about this build on the competition hardware comes from a single validation, two smoke games of 20 plies. Every rated round is another sample and the `init` and per-move lines of each are read.

Stage 1 is done. It was defined as: perft agreement with python-chess to depth 4 on the standard positions and depth 3 on every collected opening and ≥ 2000 random positions; identical legal-move sets on $\geq 100\,000$ random positions; Stage 0’s fuzz and property tests passing unchanged; import time from the zip ≤ 40 s; and ≥ 300 games against the previous version with the 95 % interval entirely above zero (source: brief, §5). All of those are met: the gates are in Section 10, import from the zip is 18.5 s, and the match was +560 Elo over 300 games with the interval far from zero.

What is next, in the order the evidence ranks it. First, **king danger** — the one outstanding gap with a lost game behind it rather than a hunch, and the reason round 70 was lost. Second, **time management**: the platform’s own logs show per-move times falling into two lumps, an iteration that finished early or one that ran to the hard ceiling, with nothing between, and the divisor that shares the clock across the game is wrong in both directions — a 23-move game ended with 69 % of its clock unspent while a 113-move game finished on 4.7 s. Third, **a full-sample re-rating** of the shipped build.

Both of the first two are now written and sit on the development branch — the timing rule of Section 7 and the king-danger term above. **Neither has yet won a promotion match against v1.0**, and under the rule this project runs on, a change that has not won one is not a version. Until that match is played and its interval is above zero, v1.0 remains the build, and the numbers in this report are its numbers. Recording that plainly matters more than the two changes do: the rule is only worth anything if it also binds the changes we happen to believe in. Not on the list: Texel tuning, which was tried and lost; and playing for the opponent’s flag, which was investigated by Yan Beletskiy and rejected on arithmetic — with a 0.5 s increment the opponent’s measured net drain was 0.063 s a move, so flagging from 10 s would take about 157 moves and the referee’s 600-ply draw lands first (source: handoff/FINDING-flagging.md). Both are recorded as negative results rather than dropped, because the reason not to do them is the useful part.

Addendum: build v0.2

After the Stage 0 build was frozen, the search and evaluation were extended within the same python-chess architecture (the compiled Stage 1 engine was deferred). v0.2 adds, each behind a plain constant so it can be bisected: null-move pruning (minimum depth 3, $R = 2 + d/6$, never in check, only with non-pawn material, with a static-evaluation guard), late-move reductions (quiet moves after the first three at depth ≥ 3 , reduced by one ply and re-searched on a fail-high), aspiration windows at the root from depth 4 (± 40 centipawns, widened fourfold on failure), futility pruning of quiet moves at depths 1–2 (margins 150/300), delta pruning in quiescence (margin 200), staged move generation, and evaluation terms for passed, doubled and isolated pawns, the bishop pair, rooks on open and semi-open files and the middlegame king pawn shield. Every new constant is a hand-chosen textbook-magnitude value recorded in docs/PROVENANCE.md. Nodes to depth 6 from the standard start fell from 238 000 to 17 000; in three seconds the engine now reaches depth 9 where v0.1 reached depth 6 (source: DECISIONS.md, 2026-09-07 v0.2 entries).

match	clock	games	+W	= D	– L	score	Elo (95%)
v0.2 vs v0.1	10 s + 0.1 s (12 workers)	300	+225	=27	–48	79.5% $\pm$ 4.2%	+235 (+193 to +285)
v0.2 vs v0.1	120 s + 0.5 s (4 workers)	60	+48	=6	–6	85.0% $\pm$ 8.2%	+301 (+208 to +454)
v0.2 vs random	3 s + 0.05 s (12 workers)	100	+100	=0	–0	100%	—

Table 9: v0.2 against the frozen v0.1 and the random baseline (source: RESULTS.md, rows v0.2-vs-v0.1-10s, v0.2-vs-v0.1-real, v0.2-fuzz-random-3s). Both promotion intervals lie above zero.

Zero failed terminations in all 460 games; the zip (nine files, 105 680 bytes unzipped) passes the packaging smoke games. The lowest clock reached at the real clock was 3.6 s, in a 242-ply fifty-move draw; the time constants are unchanged until the first validation log arrives.

Addendum: build v1.0, the compiled engine

Stage 1 was built and shipped. The board, move generation, evaluation and search were rewritten onto a 0x88 mailbox representation with integer-coded moves and compiled by numba at import; the algorithms are those of Section 5 and Section 6, unchanged in substance. python-chess remains in `agent.py` as the legality oracle and the fallback, so the safety argument of Section 9 is untouched: the compiled search proposes, and a python-chess legality check disposes.

What stayed in Python, and why. The iterative-deepening root, the aspiration window and the root move loop are Python; only `negamax` crosses into compiled code, once per root move per iteration. The boundary costs 4.0 μ s per crossing, which is negligible at a handful of crossings per move and ruinous at millions — and compiling the root as well was measured at 34.2 s of extra start-up for no gain (source: RESULTS.md, “Stage 1 phase 2” tables). Two constants changed as a direct consequence of compilation: a clock read from inside jitted code costs 301 ns through `numba.objmode` rather than about 60 ns, so `NODE_CHECK_INTERVAL` is 512 in the compiled search where it is 128 in the Python one — the same wall-clock granularity at thirteen times the node rate.

The oracle. The Python evaluation is the specification and the compiled one is a port of it, so `tests/test_fasteval.py` requires the two to agree *integer for integer* — there is no rounding to excuse a difference, and one centipawn of drift would change which move the search picks. The gate compares 2000 positions drawn from playouts of the 219 curated opening positions by default, and 20 000 when `LETAL_FULL_GATES=1`, which is what is run before anything is promoted; it also compares all 219 openings exactly, a hand-built position for each evaluation term, and every position against its colour-swapped mirror, where an indexing error in the Black tables would show up.

match	clock	games	+W	= D	- L	score	Elo (95%)
v1.0 vs v0.2	10 s + 0.1 s (8 workers)	300	+279	=19	-2	96.2% $\pm$ 1.6%	+560 (+495 to +660)
v1.0 vs Stockfish 1800	120 s + 0.5 s	16	+13	=1	-2	84.4% $\pm$ 17.3%	—
v1.0 vs Stockfish 2000	120 s + 0.5 s	16	+11	=2	-3	75.0% $\pm$ 20.0%	+191 (+35 to +512)
v1.0 vs Stockfish 2200	120 s + 0.5 s	16	+10	=1	-5	65.6% $\pm$ 23.2%	+112 (-53 to +360)
v1.0 vs Stockfish 2400	120 s + 0.5 s	16	+8	=2	-6	56.2% $\pm$ 23.5%	+44 (-125 to +238)
v1.0 vs random	3 s + 0.05 s (8 workers)	200	+200	=0	-0	100%	—

Table 10: v1.0 against the frozen v0.2, against rating-limited Stockfish at the full time control, and against the random baseline (source: RESULTS.md, rows `numba-vs-v0.2-10s`, `v1.0-vs-sf*-real`, `numba-fuzz-random`). The promotion interval against v0.2 lies far above zero. The Stockfish rows are 16 games each and their intervals are correspondingly wide: they bracket the build, they do not rate it.

Validation on the competition platform. v1.0 was uploaded and validated: verdict *valid*, two smoke games of 20 plies, one per colour. It reached **depth 10–12** at 290–569 thousand nodes per second on the competition core, against the interpreted v0.2.1’s depth 5–8 at 21.6–27.3 thousand (source: CALIBRATION.md, 2026-09-08, v1.0 validation).

A full rated game, with the per-move search table. Round 74 (Black against Zagreus 5.0, Caro-Kann Advance, won by checkmate) was played by v1.0 and the platform’s own per-move log is stored (source: handoff/LOG-round-74-zagreus.md). Seventy of our moves, 145.2 s used of a

155 s budget, 9.8 s left. It is the strongest platform evidence in the repository, because unlike the validation it is a real game against a rated opponent rather than twenty plies of smoke:

quantity	median	range
depth	11	1–16
node rate	505 k	56 k–646 k
time used ÷ soft budget	0.65	mean 0.82

Table 11: Round 74, 70 moves, recomputed from the stored log. The depth-1 and 56 k-nps entries are the final forced mate (`d1=Q, Qd4+, Qd6#`), found instantly and spending 2–8 ms of a 1.1 s budget — not a collapse under time pressure: the clock *rises* across those moves because the increment exceeds what is spent.

Two things this settles that the smoke games could not. ****The hard deadline holds on the platform.**** Two of the seventy moves exceeded it, both by exactly 1 ms — the clock is read every 512 nodes, so a millisecond is the granularity of the measurement rather than a breach. Across a full rated game the abort path did what Section 9 claims for it. And ****the soft budget does not hold, by design and by a wide margin****: 19 of 70 moves ran past it into the hard window. That is the bimodal spending Section 7 describes, measured on the shipped build in a rated game, and it is the defect the successor’s iteration control targets. That is the only platform run of the compiled build to date; every other platform figure in this report is the interpreted engine and is labelled as such.

The start-up risk, stated plainly. numba compilation is paid once, at import, inside the 90 s budget and before the clock starts. In those two smoke games it took 34.8 s and 49.4 s, for total start-up times of 35.8 s and 50.7 s. Both completed every warm-up phase, so the degraded path — in which the warm-up runs out of its 70 s budget and the remaining compilation is finished on the first move — has never yet run outside a test. The mean is comfortable; **the variance is the risk**. Identical code on identical hardware started up fifteen seconds apart, and a run slower than the worse of the two begins skipping phases. Two samples is not a distribution, so the `init` line of every rated log is watched as a further sample. This is the largest open risk in the build and it is why the warm-up deadline exists.

A Provenance of every constant

Copied from two files: `docs/PROVENANCE.md`, the human-readable record, and `weights/PROVENANCE.json`, the machine-readable one, which is the copy that **ships in the zip**. The brief requires both to cover every parameter – piece values, piece-square tables, evaluation weights, the search margins (null-move, the LMR settings, futility, aspiration, delta), the time-management constants and the transposition-table size. The shipped JSON carries 44 such rows; `docs/` never ships, so it is the JSON that answers the question a judge is most likely to ask about a constant, and it is complete rather than restricted to the tables. The generator commit is 4109710 (full hash in the JSON).

What identifies these tables is the content hash, not the commit stamp. `weights/pst.json` records `_provenance.tables_sha256` over the tables themselves, and that is the figure to check: it changes if and only if a number changes. The `git_commit` beside it records which *run* of the generator last wrote the file, which is a weaker thing — re-running `gen_pst.py` after an unrelated change re-stamps it with whatever `HEAD` then is, on tables that did not move. The two are easy to confuse and only one of them is evidence. Anyone verifying

this should re-run the generator and compare the tables, not the file: the tables reproduce bit for bit, and we have checked that they do at the commit named above.

Table 12: Provenance table. Rules: every generating script is deterministic and re-runnable; a hand-adjusted value says so in its note with the reason; a weight file’s header names its script and data hash.

parameter / file	value or shape	produced by	data	date	note
piece values (mg, eg)	P 100, N 320, B 330, R 500, Q 900, K 0	<code>tools/gen_pst.py</code> @ 4109710	none (text-book)	2026-09-06	universal textbook values, recorded as such; to be tuned in Stage 2
<code>weights/pst.json</code> PSTs	6 pieces $\times$ 64 $\times$ {mg, eg}; ranges in Section 6	<code>tools/gen_pst.py</code> @ 4109710 (parametric geometric prior; parameters in the file’s <code>_provenance</code>)	none (formula)	2026-09-06	not copied from any engine; printed and eyeballed; one coincidental row vs CPW (Section 2)
mop-up weights	edge 10, close 12	<code>tools/gen_pst.py</code>	none	2026-09-06	chosen by hand from the geometric idea
phase weights	N 1, B 1, R 2, Q 4 (total 24)	<code>tools/gen_pst.py</code> $\rightarrow$ <code>pst.json</code> (<code>evaluation.py</code> only validates them)	none	2026-09-06	standard tapered-evaluation weighting
MATE_SCORE,	100 000, 99 000	code	none	2026-09-06	arbitrary large constants
MATE_THRESHOLD	250 000 (default of Engine); dict, cleared when full and at move start above 60 %	code	none	2026-09-07	bounds memory: 120 MB RSS at the cap; int-only entries keep gc pauses $\leq$ 14 ms
TT entry cap	(TT_CLEAR_FRACTION)				

parameter / file (continued)	value or shape	produced by	data	date	note
NODE_CHECK_INTERVAL	128 (Python) / 512 (compiled)	code	none	2026-09-07	clock cadence. The two engines deliberately differ , the only shared search constant that does: at 505 k nodes/s against 55 k, both are about one millisecond, and 512 is what makes the 1 ms hard-deadline overshoot measured in round 74 the granularity rather than a breach. <i>Known defect:</i> <code>weights/PROVENANCE.json</code> still records 128 for the compiled search, because correcting it means regenerating a frozen file; cite <code>fastsearch.py:164</code> . 1024 gave 25 ms mean / 65 ms max overshoot locally; a read costs about 60 ns
MAX_PLY	128	code	none	2026-09-06	recursion bound for search, quiescence and extensions
check extension	+1 ply when in check	code	none	2026-09-06	textbook; applied before the TT probe
QS_EVASION_PLIES	4	code	none	2026-09-07	keeps eight-queens-a-side positions under 100k nodes at depth 1
DRAW_TIEBREAK_MARGIN	300	code	none	2026-09-07	“clearly ahead” = at least a minor piece
history heuristic bonus	depth ²	code	none	2026-09-06	textbook
GAME_PLY_CAP	600	code	none	2026-09-06	the referee’s cap

parameter / file (continued)	value or shape	produced by	data	date	note
time constants	Section 7 table	code	platform rated logs and the v1.0 valida- tion, 2026-09- 08	2026-09-08	brief §6.2 gave the initial values; calibrated against the platform's own per-move timing on 2026-09-08 (source: CALIBRA- TION.md)